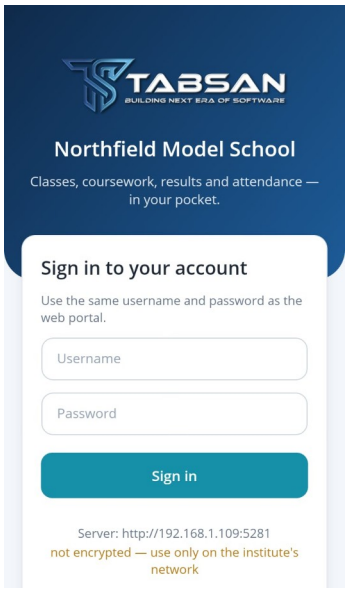
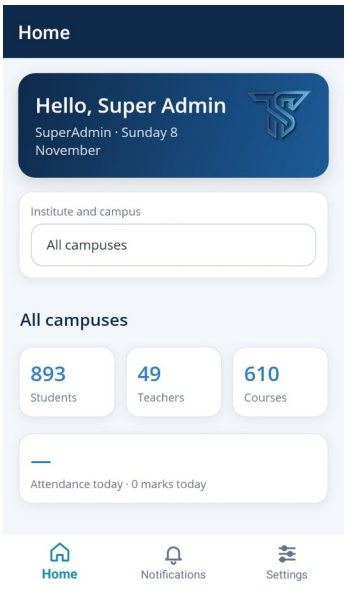
TABSAN EDUSPHERE

Installation and Connection Guide

Database, server, web portal and mobile app — how they connect, and how to add an institute with its own database



EduSphere
one app for every institute



Version 1.0 · 8 November 2026

Tabsan Nexora — Building the Next Era of Software

About This Guide

This guide explains how the parts of Tabsan EduSphere are connected: the **database**, the **API** (the server program that holds the rules and opens the database), the **web portal** and the **mobile app**. It shows where each connection is set, how to install a new institute with its own database, how to point the portal at its API, and how phones are connected to the right server.

Everything in it was done for real while the guide was written. A second, independent installation called **Northfield Model School** was set up next to an existing one on the same Windows 11 PC. It got its own database on the same SQL Server Express and its own API and portal on different ports. Then the Android app on a phone emulator was moved from the first institute to the second. Chapter 6 lists every step of that example; the other chapters explain each step in general and use its screenshots.

The example runs in the **Development** environment on one PC, as a trial or pilot would. A live installation runs in **Production**, which needs a few more settings (Redis, secrets, HTTPS). Chapter 7 lists them, as the program checks them at start-up.

Item	Detail
Product	Tabsan EduSphere: API and web portal (.NET 8), mobile app 1.0.0 (standard build)
Document	Installation and Connection Guide: database, server, portal and mobile app
Version	1.0 — 8 November 2026
Audience	IT staff who install EduSphere or add an institute (chapters 1–7); helpdesk staff (chapter 8)
Example used	Two institutes on one PC: the main installation (API port 5181, portal 5063, database Tabsan-EduSphere) and Northfield Model School (API 5281, portal 5163, database EduSphere-Northfield), both on SQL Server Express; phones reach the PC at 192.168.1.109
Related documents	Mobile App Connection Guide (everything about the app) · SuperAdmin Guide · License KeyGen Guide · SSO Setup Guide
Publisher	Tabsan Nexora

> **NOTE** Addresses such as `192.168.1.109`, ports such as `5281` and names such as `EduSphere-Northfield` are those of the example. Use your own server's address, the ports you choose and your institute's database name.

© 2026 Tabsan Nexora. All rights reserved.

Contents

About This Guide 2
Contents 3
1 How the Pieces Connect 5
1.1 Four parts, three connections 5
1.2 What each part must be told 5
1.3 Where the programs read their settings 6
2 Prepare the Database 8
2.1 Choose the SQL Server 8
2.2 Give the API the right to use its database 8
2.3 Write the connection string 9
2.4 Give the API its connection string 9
2.5 What the first start does 11
3 Install and Start the API 12
3.1 What the server needs 12
3.2 Copy the API to the server 12
3.3 Start the API 13
3.4 Check that the API works 13
3.5 Keep the API running 14
4 Install and Connect the Portal 15
4.1 Copy the portal to the server 15
4.2 Point the portal at the API 15
4.3 Sign in for the first time 16
4.4 Give the institute its name 16
4.5 Upload the licence 17
4.6 Set up the institute 18
5 Connect the Mobile App 19
5.1 Tell the API where phones open the portal 19
5.2 Let phones reach the server 19
5.3 Give people the API address, not the portal address 20
5.4 Connect and sign in 20
5.5 Move a phone to another institute 21
6 Worked Example: a Second Institute on the Same Server 23
6.1 The plan 23
6.2 Step by step 23
6.3 The result 24
6.4 Keeping installations apart 24
7 Live Installations (Production) 26
7.1 Settings the API requires in Production 26
7.2 Settings the portal requires in Production 26
7.3 The listening address in Production 27
7.4 HTTPS and a public address 27
7.5 Protect the Database Setup page 27
7.6 Keep your settings safe across updates 28

© 2026 Tabsan Nexora. All rights reserved.

8 Troubleshooting....29
8.1 Database and API....29
8.2 Portal....29
8.3 Mobile app....30
Appendix A — Settings Reference....31
Appendix B — Addresses, Ports and the App....32
Appendix C — Document History....33

© 2026 Tabsan Nexora. All rights reserved.

1 How the Pieces Connect

Before installing anything, it helps to see which part talks to which. There are only three connections to set, and the database is behind exactly one of them.

1.1 Four parts, three connections

The **API** (`Tabsan.EduSphere.API`) is the heart of an installation: it holds the rules, runs the background jobs and is the only part that opens the database. The **web portal** (`Tabsan.EduSphere.Web`) shows pages in the browser and gets all its data from the API. The **mobile app** also talks to the API directly; it opens the portal only for quizzes, some materials and single sign-on.

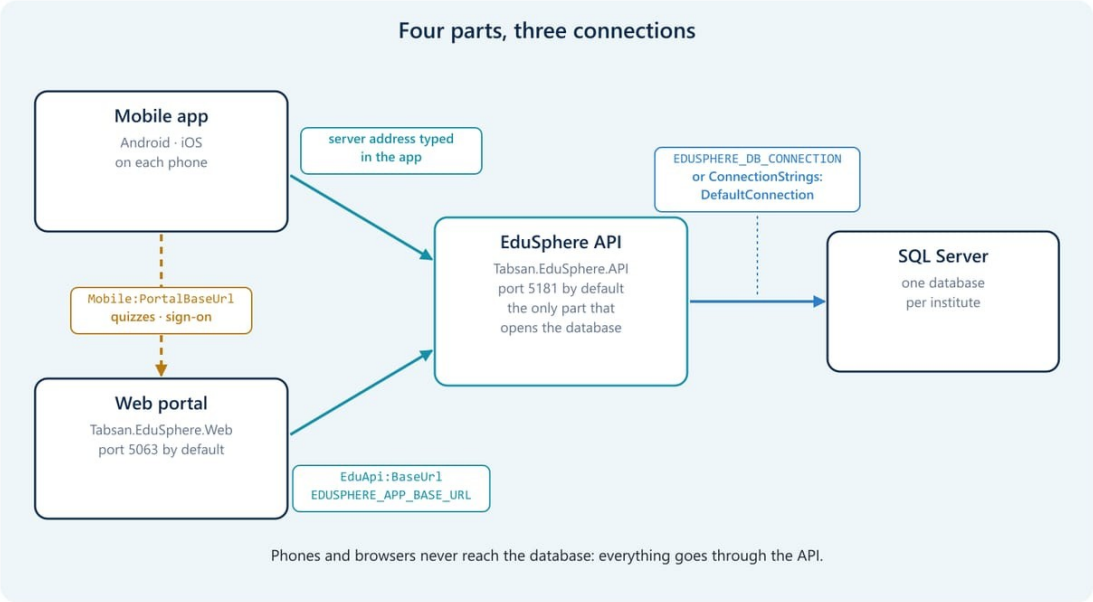


Figure 1.1 *The four parts, and the setting behind each connection*

Part	What it is	Default address	Talks to
Database	A SQL Server database, one per institute	the SQL Server, e.g. `localhost\SQLEXPRESS`	nothing (the API comes to it)
API	`Tabsan.EduSphere.API.exe`	`http://localhost:5181`	the database
Web portal	`Tabsan.EduSphere.Web.exe`	`http://localhost:5063`	the API
Mobile app	EduSphere for Android and iOS	on each phone	the API (and the portal for quizzes and sign-on)

1.2 What each part must be told

Each arrow in Figure 1.1 is one setting. If one is wrong, that part either does not start, or quietly talks to the wrong place: for example, a second institute's portal showing the first institute's data. Chapter 6 shows how this happens and how to avoid it.

© 2026 Tabsan Nexora. All rights reserved.

Who	Must know	Setting (recommended first)	Where
API	the database	`EDUSPHERE_DB_CONNECTION`, or `ConnectionStrings:DefaultConnection`, or the Database Setup page	on the server, before the API starts
Web portal	the API's address	`EDUSPHERE_APP_BASE_URL`, or `EduApi:BaseUrl`	on the server, before the portal starts
API	the portal address phones can open	`Mobile:PortalBaseUrl`	on the server
Each phone	the API's address	typed in the app once (or built into a branded app)	on each phone

1.3 Where the programs read their settings

The API and the portal read settings from several places. When the same setting appears in more than one, the first one in this list wins.

The database (API)

1. **The Database Setup page's saved settings** (`App_Data\database-setup.json` in the API's folder, encrypted). When this file exists it overrides everything below (section 2.4).
2. **The environment variable** `EDUSPHERE_DB_CONNECTION`. The program also accepts `EDUSPHERE_DEFAULT_CONNECTION`, `DB_CONNECTION` or `DATABASE_URL`.
3. **A profile in** `environments.json`, when `ASPNETCORE_ENVIRONMENT` or `EDUSPHERE_ENVIRONMENT` names one. Developers' copies have this file (`src/environments.json`); published folders do not.
4. **The setting** `Deployment:Database:ConnectionString`, then `ConnectionStrings:DefaultConnection`. Each can be given on the command line (`--ConnectionStrings:DefaultConnection=…`), as an environment variable (`ConnectionStrings__DefaultConnection`) or in a settings file (`appsettings.Local.json`, then `appsettings.Production.json` or `appsettings.Development.json`, then `appsettings.json`).

The API's address (portal)

1. **The environment variable** `EDUSPHERE_APP_BASE_URL`. The program also accepts `EDUSPHERE_APP_CONNECTION` or `APP_CONNECTION_STRING`.
2. **A profile in** `environments.json`, as above (its `AppConnectionString`).
3. **The setting** `EduApi:BaseUrl`, given on the command line, as an environment variable or in a settings file.

© 2026 Tabsan Nexora. All rights reserved.

> **IMPORTANT** Prefer the environment variables `EDUSPHERE_DB_CONNECTION` (API) and `EDUSPHERE_APP_BASE_URL` (portal). They win over every settings file and profile, so a second installation can never pick up the first one's database by accident. Set them **for that program only** (in its start script, or in its IIS site), never for the whole computer: a machine-wide variable would point every installation on the server at the same database.

Both programs write, in their first lines, what they actually used. Check these lines after every change (Figures 3.1 and 4.1).

© 2026 Tabsan Nexora. All rights reserved.

2 Prepare the Database

Each institute has its own database. The API creates the tables itself the first time it starts, so preparing the database mostly means choosing the SQL Server and giving the API the right to use it.

2.1 Choose the SQL Server

Who does this: IT administrator or database administrator

- **SQL Server Express**: free, and enough for a school or a small college (up to 10 GB per database). The example uses `localhost\SQLEXPRESS` with Windows authentication.
- **SQL Server Standard or Enterprise** on the same server or a separate database server: for larger institutes and several institutes on one server.
- **Azure SQL Database**: when the API runs in the cloud.
- **LocalDB** is for developers only. It runs per Windows user and stops when that user signs out.

One SQL Server can hold the databases of several institutes. Each installation of the API uses exactly one database (Figure 6.1).

2.2 Give the API the right to use its database

Who does this: Database administrator

The API needs to create and change tables on its first start and after updates, so give its sign-in **db_owner** on its own database, and nothing on any other database.

How the API signs in to SQL Server	Who needs the rights
Windows authentication (`Trusted_Connection=True`)	The Windows account the API runs as. When it runs in a console window, that is the person signed in. In IIS it is the site's application pool, for example `IIS APPPOOL\EduSphere-Northfield-API`: create a SQL login for it.
SQL authentication (`User Id=…;Password=…`)	That SQL login. Use a separate login per institute, for example `edusphere_northfield`.

There are two ways to create the database:

- **Let the API create it** (the example did this). The API's sign-in also needs the right to create databases (the **dbcreator** server role) on the first start only.
- **Create an empty database first** (for example in SQL Server Management Studio) and give db_owner on it. The API then creates the tables in it. A DBA who wants to see the tables before the API runs can instead run `Scripts/01-Schema-Current.sql` (the same changes as the API's migrations, safe to run twice).

© 2026 Tabsan Nexora. All rights reserved.

2.3 Write the connection string

The connection string tells the API which SQL Server and database to use and how to sign in. It is one line, without spaces around `=` or `;`. Below it is split after `;` only to fit the page.

SQL Server Express on the same computer, Windows authentication (the example)

```
Server=localhost\SQLEXPRESS;
Database=EduSphere-Northfield;
Trusted_Connection=True;
MultipleActiveResultSets=true;
TrustServerCertificate=True
```

SQL Server on a database server, SQL login

```
Server=db01.school.local,1433;
Database=EduSphere-Northfield;
User Id=edusphere_northfield;
Password=(the login's password);
MultipleActiveResultSets=true;
Encrypt=True
```

Azure SQL Database

```
Server=tcp:northfield-sql.database.windows.net,1433;
Database=EduSphere-Northfield;
User Id=edusphere_northfield;
Password=(the login's password);
MultipleActiveResultSets=true;
Encrypt=True
```

Part	Meaning
`Server`	The SQL Server: a name, `name\instance` for a named instance such as SQLEXPRESS, or `name,port`.
`Database`	The institute's database. A different name for every institute.
`Trusted_Connection=True`	Windows authentication: no password in the string.
`MultipleActiveResultSets=true`	Needed by EduSphere. Always include it.
`TrustServerCertificate=True`	Accept the SQL Server's own certificate. Use it only for a SQL Server on the same computer or a trusted network; for other servers install a proper certificate on SQL Server and leave this out.

2.4 Give the API its connection string

Recommended: an environment variable

Set `EDUSPHERE_DB_CONNECTION` in the window or IIS site that starts the API (section 3.3). It wins over every settings file, and it keeps the password out of files in the program folder.

© 2026 Tabsan Nexora. All rights reserved.

Or: the Database Setup page

The API has its own setup page at `http://<API address>/setup`. It opens by itself when the API cannot reach its database. You choose SQL Server Express or another server, type the server and database, and use **Test Connection** (Figure 2.1). **Save & Continue** stores the settings encrypted in `App_Data\database-setup.json` in the API's folder. They take effect at once and override every other setting.

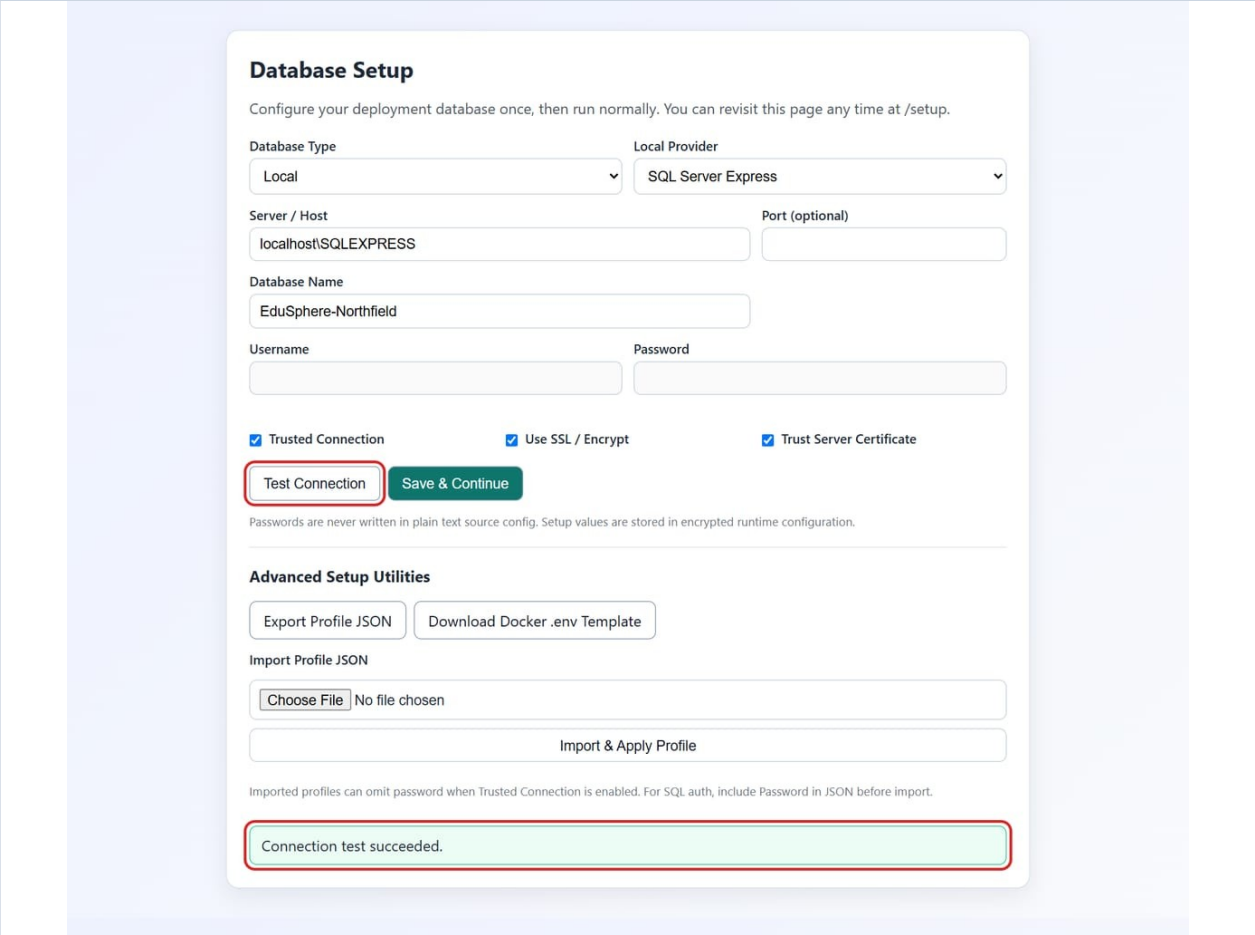


Figure 2.1 *The API's Database Setup page: the example's database tested (not saved)*

> **WARNING** The Database Setup page asks for **no sign-in**, also after set-up. Anyone who can reach the API can open it and change the database the API uses. Use environment variables instead where you can, and block the page once the installation works (section 7.5).

Or: a settings file

Put the connection string in `appsettings.Local.json` in the API's folder. An update does not overwrite this file, because it is never part of the published program.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost\\SQLEXPRESS;Database=EduSphere-Northfield;…"
  }
}
```

© 2026 Tabsan Nexora. All rights reserved.

Write the whole connection string from section 2.3 on that one line. In JSON files every backslash is written twice (`localhost\\SQLEXPRESS`).

2.5 What the first start does

When the API starts and the database does not exist yet, it does not stop: it creates the database and fills in everything a new institute needs. In the example this took **16 seconds** and created 146 tables (88 migrations).

- Creates the database and all tables.
- Adds the roles (SuperAdmin, Admin, Faculty, Student, Finance, Parent), the modules, the menus and the report list.
- Adds a default institute (tenant) and campus, named **Default Tenant** and **Main Campus**.
- Creates the first **SuperAdmin** account from these environment variables:

Variable	Meaning
`TABSAN_SUPER_USERNAME`	Username of the first SuperAdmin; `superadmin` when not set.
`TABSAN_SUPER_PASSWORD`	Its password: **required** on the first start. Choose one that meets the portal's password rules (at least 12 characters, upper and lower case, a digit and a symbol). Without it the API stops with "TABSAN_SUPER_PASSWORD environment variable is required for initial seeding."
`TABSAN_SUPER_EMAIL`	Optional email address of the account.

These variables are read only while the database has no SuperAdmin. Remove them after the first start. People change their own password in the portal.

sqlcmd -S **localhost\SQLEXPRESS** -E -C -Q "SELECT ... FROM sys.databases"

```
Database              Created           SizeMB
--------------------  ----------------  ------
Tabsan-EduSphere      2026-11-07 03:17  400
EduSphere-Northfield  2026-11-08 02:24  80
```

Figure 2.2 *One SQL Server Express with both institutes' databases (the example's new database was created by the API)*

> **NOTE** Builds before 8 November 2026 created the SuperAdmin only on the **second** start of a new database. If you cannot sign in after the very first start of an older build, restart the API once.

© 2026 Tabsan Nexora. All rights reserved.

3 Install and Start the API

3.1 What the server needs

Who does this: IT administrator

- Windows Server 2019 or later, or Windows 10/11 for a trial; Linux works as well.
- The **.NET 8 ASP.NET Core Runtime**. On Windows with IIS, install the **.NET 8 Hosting Bundle**, which includes it.
- Network access to the SQL Server (TCP 1433, or the named instance's port with the SQL Server Browser service).
- For a live installation: a Redis server (or Azure Cache for Redis) and a certificate for HTTPS (chapter 7).
- Free ports for the API and the portal. Defaults are 5181 and 5063; every further installation on the same server needs two other ports (the example uses 5281 and 5163).

3.2 Copy the API to the server

Who does this: Tabsan or the institute's IT team

The API is published from the source code into a folder, and that folder is copied to the server. Use one folder per institute:

```
dotnet publish src/Tabsan.EduSphere.API/Tabsan.EduSphere.API.csproj `
  -c Release -o C:\EduSphere\Northfield\api
```

- Publish a new installation into an **empty** folder.
- Keep your own settings out of `appsettings.json`, which an update replaces. Use environment variables or `appsettings.Local.json`.
- The folder holds `App_Data` (the Database Setup page's file, if used) and `uploads` (files people upload, unless they go to Azure Blob or S3). Two institutes must never share a folder.

> **NOTE** Builds before 8 November 2026 had a fault: publishing again into a folder that already held the API (an update) replaced its `appsettings*.json` with those of the background-jobs program. The API then answered every request with error 500. If that happened, delete the `appsettings*.json` files in the folder and publish again.

© 2026 Tabsan Nexora. All rights reserved.

3.3 Start the API

In a PowerShell window on the server (for a trial; section 3.5 shows how to keep it running):

```
cd C:\EduSphere\Northfield\api
$env:ASPNETCORE_ENVIRONMENT = "Development"    # trial; live: chapter 7
$db  = "Server=localhost\SQLEXPRESS;Database=EduSphere-Northfield;"
$db += "Trusted_Connection=True;MultipleActiveResultSets=true;"
$db += "TrustServerCertificate=True"
$env:EDUSPHERE_DB_CONNECTION = $db
$env:TABSAN_SUPER_PASSWORD = Read-Host "First SuperAdmin password"
.\Tabsan.EduSphere.API.exe --urls http://0.0.0.0:5281 `
  --Mobile:PortalBaseUrl=http://192.168.1.109:5163
```

Part	Why
`--urls http://0.0.0.0:5281`	Listen on port 5281 on every network card, so phones on the Wi-Fi can reach it. `http://localhost:5281` would accept only programs on the server itself.
`--Mobile:PortalBaseUrl=…`	The portal address phones can open (section 5.1).
`Read-Host`	Asks for the password, so it is not written in a script or in the window's history.

The example also added `--Licensing:EnforceReadOnlyWhenUnlicensed=false`, because a trial has no licence file. Leave it out on a real installation and upload the licence instead (section 4.5).

Northfield API — console when it starts

```
[Startup] Environment: Development | App: Tabsan.EduSphere.API
[Startup] Configuration sources: appsettings.json, appsettings.{Environment}.json, environment
variables
[Startup][Warning] Database connection string was overridden by environment variable.
[Startup] Database type: Local SQL Server
[Startup] Database connection source: Environment variable: EDUSPHERE_DB_CONNECTION
[Startup] Runtime database setup source: appsettings/environment
[02:25:09 INF] Now listening on: http://0.0.0.0:5281
```

Figure 3.1 *The first lines of the example's API: the database came from EDUSPHERE_DB_CONNECTION*

The line **Database connection source** tells you which setting the API used. If it does not name the one you set, another setting won (section 1.3).

3.4 Check that the API works

1. Open `http://<server>:5281/health` in a browser. It must say **Healthy**.
2. Open `http://<server>:5281/api/v1/mobile/config`. It must show a short JSON text with `institutionName` (Figure 5.1). This is the same check the mobile app makes.
3. Open it once more from another computer or a phone on the same network, with the server's network address instead of `localhost`. If that fails but the first two worked, the firewall is closed (section 5.2).

© 2026 Tabsan Nexora. All rights reserved.

3.5 Keep the API running

A program started in a PowerShell window stops when the window closes or the person signs out. That is fine for a trial; a live server hosts the API in **IIS** (Windows) or as a **systemd** service (Linux).

IIS (Windows)

1. Install the .NET 8 Hosting Bundle and restart IIS.
2. Create a site for the API, for example **EduSphere-Northfield-API**, with its own application pool (".NET CLR version: No Managed Code") and the published folder as its path. The folder already contains the `web.config` IIS needs.
3. Bind the site to its port (5281) or, better, to a host name with HTTPS (chapter 7).
4. Add the environment variables in **Configuration Editor → system.webServer/aspNetCore → environmentVariables**, or in `web.config`. Keep a copy of `web.config`: publishing replaces it.
5. Give the application pool's account db_owner on the database when the connection string uses Windows authentication (section 2.2).

```
<aspNetCore processPath="dotnet" arguments=".\Tabsan.EduSphere.API.dll"
            hostingModel="inprocess" stdoutLogEnabled="false">
  <environmentVariables>
    <environmentVariable name="EDUSPHERE_DB_CONNECTION" value="Server=…" />
  </environmentVariables>
</aspNetCore>
```

© 2026 Tabsan Nexora. All rights reserved.

4 Install and Connect the Portal

4.1 Copy the portal to the server

```
dotnet publish src/Tabsan.EduSphere.Web/Tabsan.EduSphere.Web.csproj `
  -c Release -o C:\EduSphere\Northfield\portal
```

The portal never opens the database: it gets everything from the API. Its folder must still be separate from every other installation's portal.

4.2 Point the portal at the API

The portal needs the address of **its** API. When the portal and the API are on the same server, use `localhost` and the API's port:

```
cd C:\EduSphere\Northfield\portal
$env:ASPNETCORE_ENVIRONMENT = "Development"    # trial; live: chapter 7
$env:EDUSPHERE_APP_BASE_URL = "http://localhost:5281"
$env:EDUSPHERE_DB_CONNECTION = $db             # the same string as the API
.\Tabsan.EduSphere.Web.exe --urls http://0.0.0.0:5163
```

- `EDUSPHERE_APP_BASE_URL` is the API the portal calls. When the API is on another server, use its address, for example `https://api.northfield.edu`.
- `EDUSPHERE_DB_CONNECTION`: the portal does not use it, but it does not start without a database setting, and it writes the source in its first lines. Give it the same string as the API, so nothing in its log points at another institute.

Northfield portal — console when it starts

```
[Web] Environment: Development | App: Tabsan.EduSphere.Web
[Web][Warning] App connection string was overridden by environment variable.
[Web][Warning] Database connection string was overridden by environment variable.
[Web] API address (EduApi:BaseUrl): http://localhost:5281
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://0.0.0.0:5163
```

Figure 4.1 *The first lines of the example's portal: its API is on port 5281*

The line **API address (EduApi:BaseUrl)** must show this institute's API.

> **IMPORTANT** Builds before 8 November 2026 used `EDUSPHERE_APP_BASE_URL` (and the `environments.json` profile) only for a start-up check; the portal's pages kept calling the API named in `EduApi:BaseUrl`. With such a build, set `--EduApi:BaseUrl=http://localhost:5281` on the command line as well. Otherwise a second institute's portal shows the first institute's data.

© 2026 Tabsan Nexora. All rights reserved.

4.3 Sign in for the first time

Open the portal at `http://<server>:5163` and sign in as **superadmin**, with the password given in `TABSAN_SUPER_PASSWORD` when the API first started.

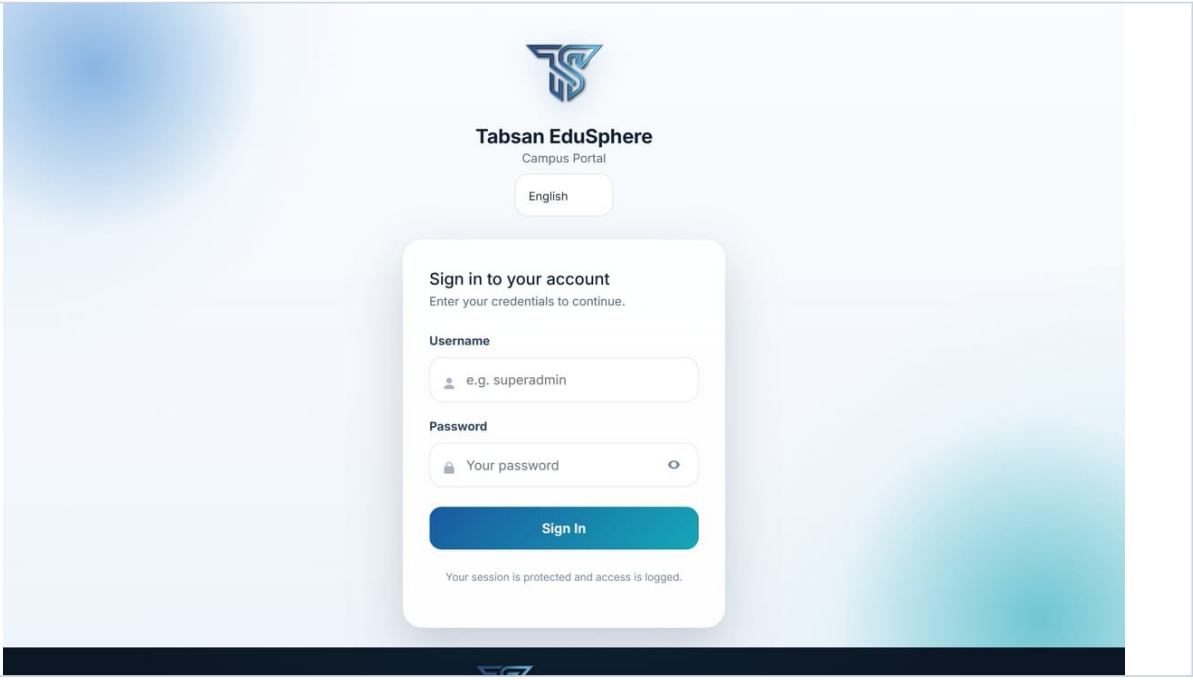


Figure 4.2 *The new institute's portal (the sign-in page always says Tabsan EduSphere)*

> **TIP** Two portals on one computer (as in the example) share their sign-in in the same browser, because browsers keep cookies per host name, not per port. Signing in to one can sign you out of the other. Use the server's name for one and its address for the other (for example `http://localhost:5063` and `http://192.168.1.109:5163`), or two different browsers. Separate institutes on their own domains are not affected.

4.4 Give the institute its name

1. Open **System Settings** → **Dashboard Settings**.
2. Type the institute's name in **University / Institution Name** and a **Portal Subtitle**, for example **School Portal**.
3. Save. The name appears in the portal after the next sign-in, and in the mobile app's sign-in screen at once.

© 2026 Tabsan Nexora. All rights reserved.

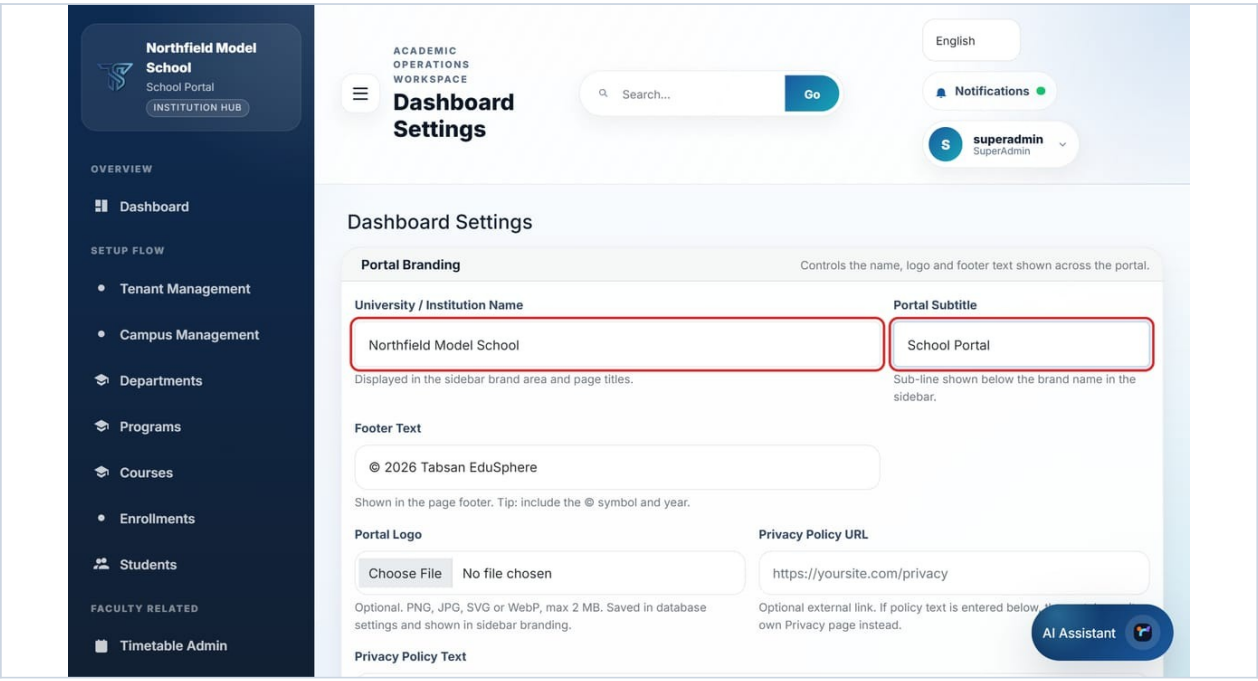


Figure 4.3 *Dashboard Settings: the institute's name and subtitle*

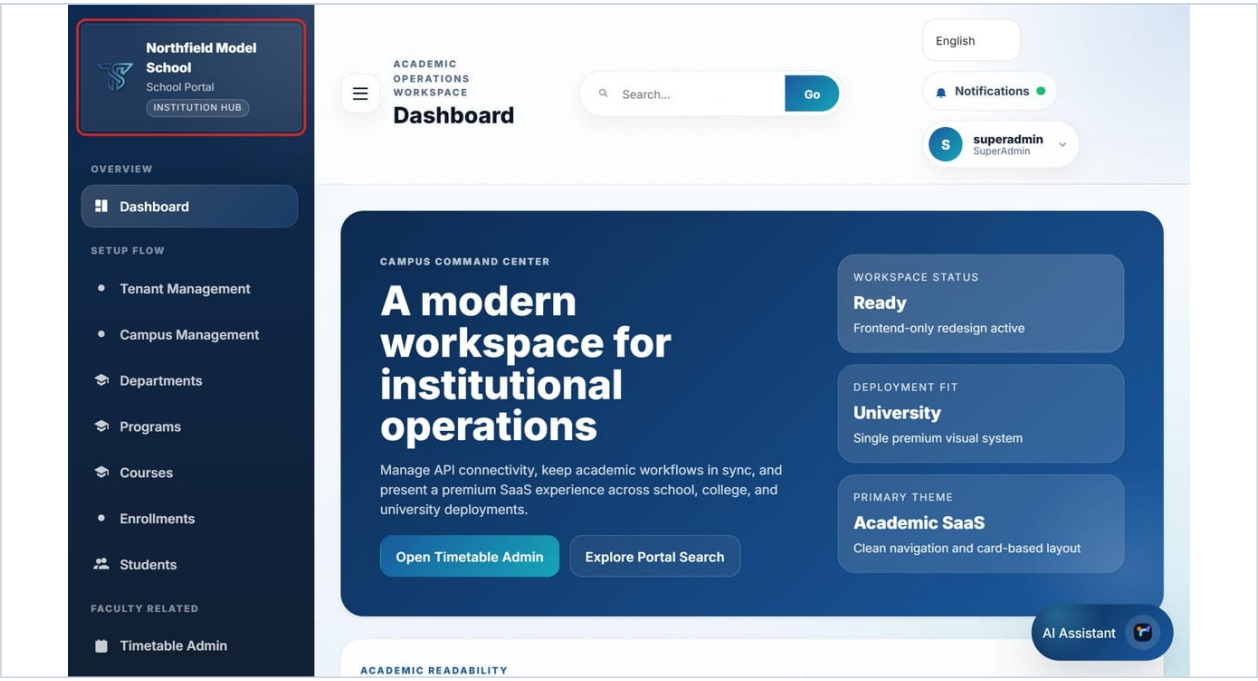


Figure 4.4 *After the next sign-in the portal carries the new name*

4.5 Upload the licence

A new database has no licence. Until one is uploaded, the API is **read-only**: people can sign in and read, but every change is refused with "The portal is in read-only mode. A Super Admin must upload a valid license under Settings → License Update."

1. Open **System Settings** → **License Update**. **Status** shows **None** on a new installation.
2. Choose the `.tablic` licence file Tabsan issued for this institute and press **Upload & Activate**.

© 2026 Tabsan Nexora. All rights reserved.

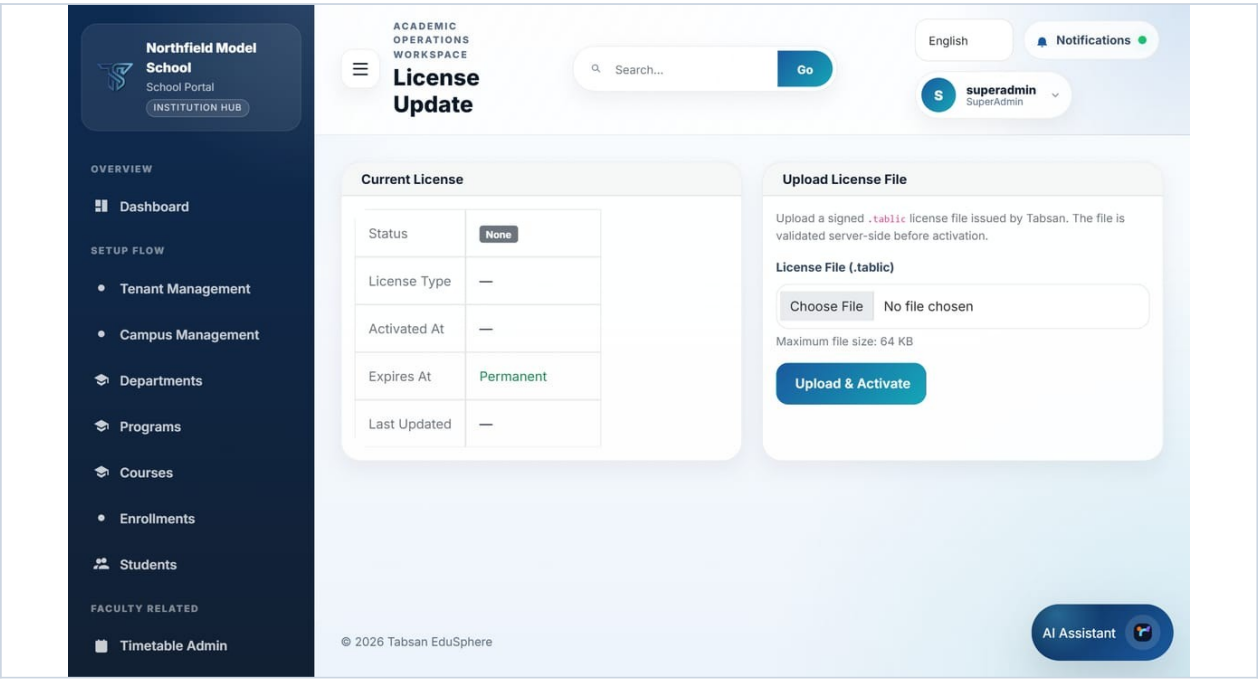


Figure 4.5 *License Update on the new installation: no licence yet*

For a trial without a licence file the API can be started with `--Licensing:EnforceReadOnlyWhenUnlicensed=false`, as the example was. Never leave that on a live server.

4.6 Set up the institute

The installation is now connected. What follows is ordinary portal work, described in the SuperAdmin Guide and the User Manual:

- **Institution Policy**: which of School, College and University this institute uses.
- **Tenant and Campus Management**: rename **Default Tenant** and **Main Campus**, or add campuses.
- Departments, programmes or classes, courses and the timetable.
- **User Import**: staff, students and parents; **Parent Links** for School parents.
- For schools, the scripts in `Scripts/Schools-And-Campuses` and `Scripts/School-Demo-Data` can fill a test copy. Pass the new database and API, for example `-Database EduSphere-Northfield -Api http://localhost:5281/api/v1`.

© 2026 Tabsan Nexora. All rights reserved.

5 Connect the Mobile App

There is one EduSphere app for every institute. It has no server built in: each person types their institute's server address once. This chapter covers what the server needs for that; the Mobile App Connection Guide covers the app itself.

5.1 Tell the API where phones open the portal

Quizzes, some course materials and single sign-on open in the portal. The API tells the app where that is with `Mobile:PortalBaseUrl` (section 3.3). Use an address phones can reach: the server's network address or its public name, never `localhost`.

Check it at `http://<server>:5281/api/v1/mobile/config`, which is what the app reads first:

GET **http://192.168.1.109:5281/api/v1/mobile/config** → 200 OK


```
{
  "institutionName": "Northfield Model School",
  "subtitle": "School Portal",
  "minimumVersion": "1.0.0",
  "latestVersion": "1.0.0",
  "androidStoreUrl": "",
  "iosStoreUrl": "",
  "portalBaseUrl": "http://192.168.1.109:5163",
  "appRedirectUri": "edusphere://sso",
  "defaultLanguage": "en",
  "languages": "[3 languages]",
  "ssoProviders": [],
  "pushEnabled": false
}
```


Figure 5.1 *The configuration the app reads: the institute's name and its portal address*

5.2 Let phones reach the server

On the institute's own network

- The API must listen on the network (`--urls http://0.0.0.0:5281`, section 3.3).
- Give the server a **fixed address** (a DHCP reservation), so the address people type keeps working.
- Open both ports in the server's firewall. In an administrator PowerShell window:

```
New-NetFirewallRule -DisplayName "Northfield API" -Direction Inbound `
  -Protocol TCP -LocalPort 5281 -Action Allow -Profile Private,Domain
New-NetFirewallRule -DisplayName "Northfield portal" -Direction Inbound `
  -Protocol TCP -LocalPort 5163 -Action Allow -Profile Private,Domain
```

- The server's network must be **Private** or **Domain**, not Public (Settings → Network & internet → the network → Network profile type).
- Phones must be on that network's Wi-Fi, or on a VPN to it. The app accepts plain `http://` only for such local addresses and marks the connection "not encrypted".

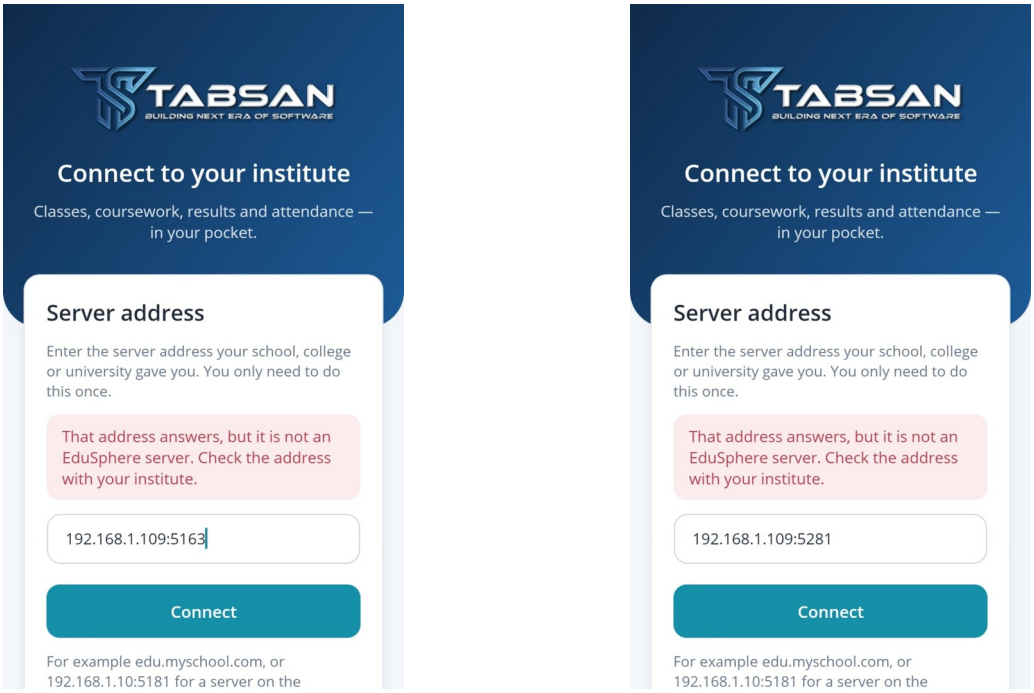
© 2026 Tabsan Nexora. All rights reserved.

On the internet

Phones reach the server through its public name with HTTPS and a trusted certificate (chapter 7). The app refuses plain `http://` to an internet address.

5.3 Give people the API address, not the portal address

People type the address of the **API**, not of the portal. In the example that is `192.168.1.109:5281`. The portal's address (`:5163`) answers too, but the app sees that it is not an EduSphere API and says so:



The portal's port: refused **The API's port: right**

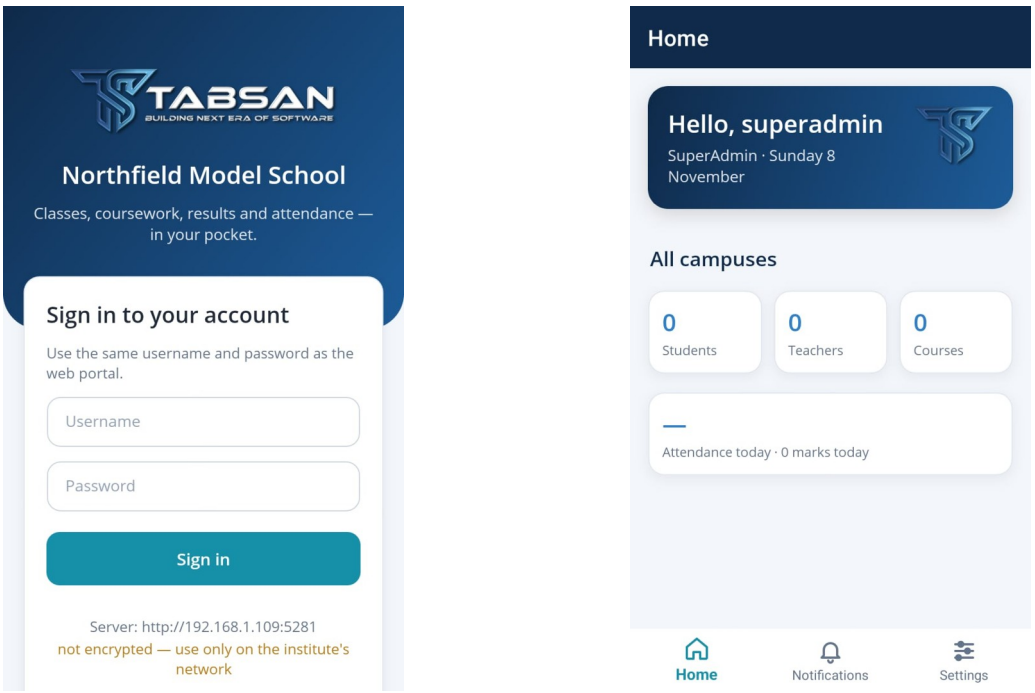
Figure 5.2 *Typing the portal's port instead of the API's, and the right address*

A short message to send to everyone works well: "Install EduSphere, then type **192.168.1.109:5281** as the server address. Sign in with your portal username and password."

5.4 Connect and sign in

1. Open the app, type the server address and tap **Connect**.
2. The sign-in screen shows the **institute's name** from Dashboard Settings, and below it the server.
3. Sign in with the same username and password as the portal.

© 2026 Tabsan Nexora. All rights reserved.



The institute's name **Its own data**

Figure 5.3 *Connected to Northfield: its name on the sign-in screen, and its new (still empty) institute on Home*

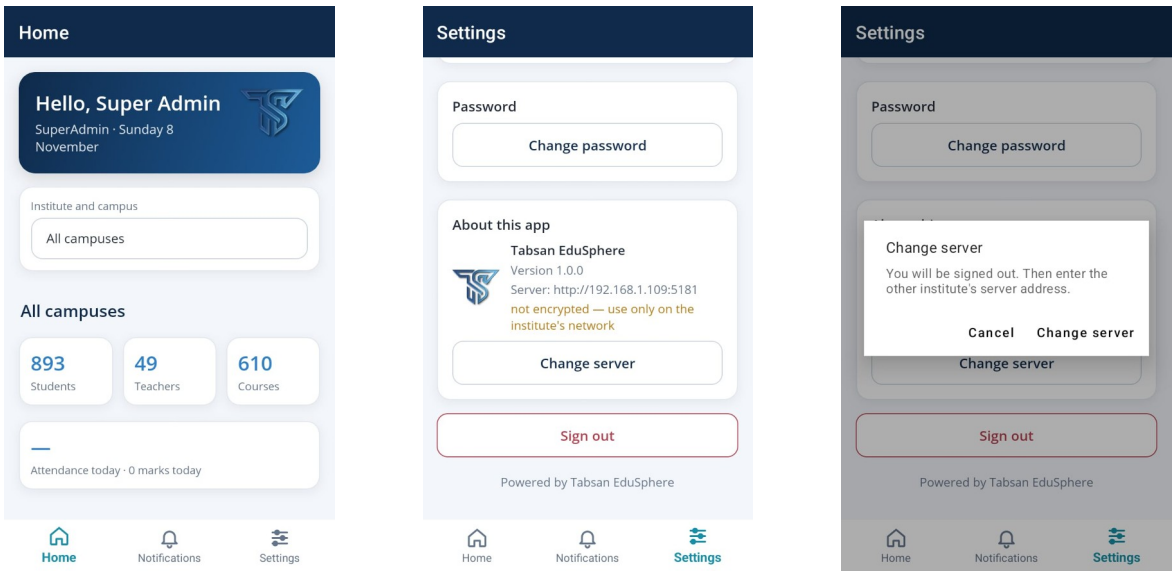
Home shows **0** students, teachers and courses, because Northfield's database is new. That is the quickest proof that the phone is on the right server: the main institute on the same PC has 893 students (Figure 5.4).

5.5 Move a phone to another institute

A phone that is connected to one institute can be moved to another without reinstalling. Accounts belong to one server, so the app signs out first.

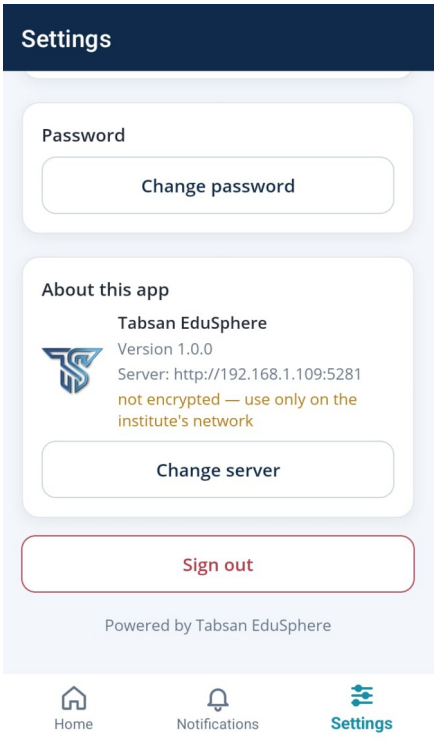
1. Open **Settings**. **About this app** shows the server in use.
2. Tap **Change server** and confirm.
3. Type the other institute's API address, tap **Connect** and sign in.

© 2026 Tabsan Nexora. All rights reserved.



Main institute **Settings → Change server** **Confirm**

Figure 5.4 *Moving the phone away from the main institute (port 5181)*



Now on Northfield

Figure 5.5 *Settings after the move: server 192.168.1.109:5281*

> **NOTE** A branded app built for one institute has its server fixed and no **Change server** (Mobile App Connection Guide, chapter 3).

© 2026 Tabsan Nexora. All rights reserved.

6 Worked Example: a Second Institute on the Same Server

This is exactly what was done for the screenshots in this guide, on 8 November 2026: a second institute added next to an existing installation, on one Windows 11 PC with SQL Server Express, and a phone moved from the first to the second.

6.1 The plan

	Institute A (existing)	Institute B: Northfield Model School
Database	`Tabsan-EduSphere`	`EduSphere-Northfield` (new)
SQL Server	`localhost\SQLEXPRESS`	the same
API	port 5181	port 5281
Portal	port 5063	port 5163
Program folder	the developers' copy	its own published folder
Phones type	`192.168.1.109:5181`	`192.168.1.109:5281`

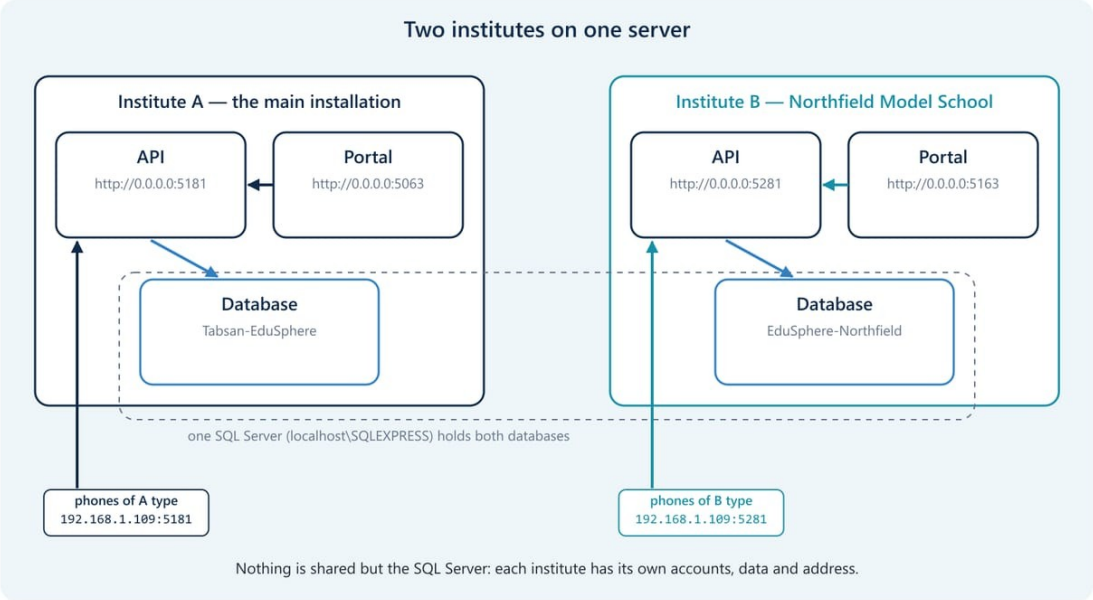


Figure 6.1 *Two independent installations sharing one SQL Server*

6.2 Step by step

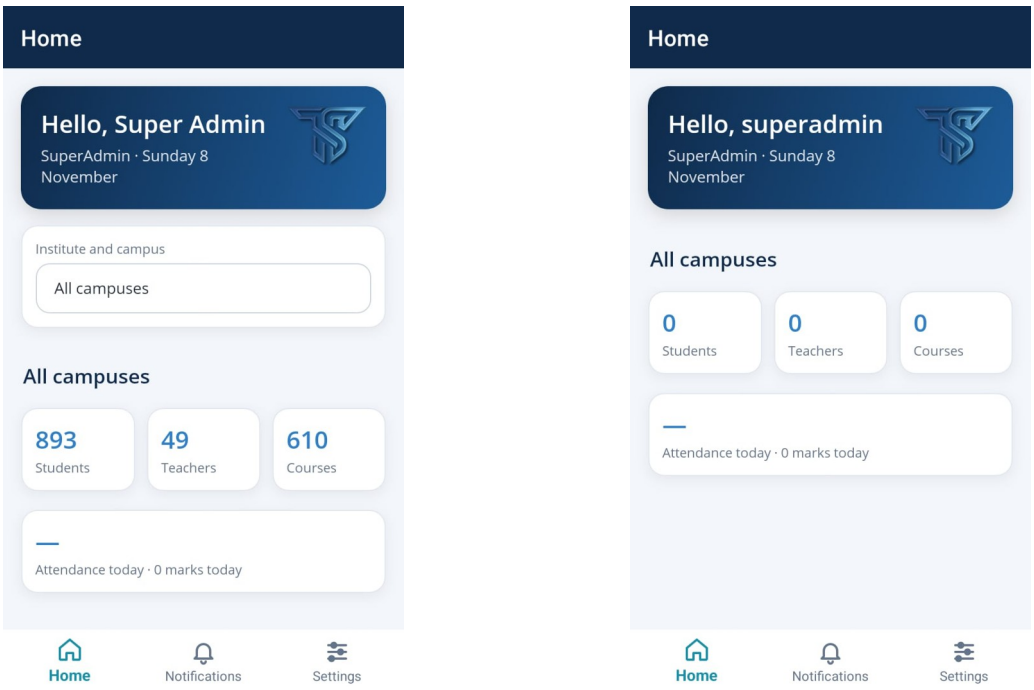
1. **Published** the API and the portal into a new folder (section 3.2).
2. **Started the API** with `EDUSPHERE_DB_CONNECTION` naming the new database, `TABSAN_SUPER_PASSWORD` for the first SuperAdmin, `--urls http://0.0.0.0:5281` and `--Mobile:PortalBaseUrl=http://192.168.1.109:5163` (section 3.3). The database did not exist; the API created it in 16 seconds and answered `/health` with **Healthy**.

© 2026 Tabsan Nexora. All rights reserved.

3. **Started the portal** with `EDUSPHERE_APP_BASE_URL=http://localhost:5281` and `--urls http://0.0.0.0:5163` (section 4.2). Its log showed every call going to port 5281.
4. **Tested the database** on the API's Database Setup page: "Connection test succeeded." (not saved, Figure 2.1).
5. **Signed in** to the new portal as superadmin, named the institute **Northfield Model School** with the subtitle **School Portal**, and looked at License Update (sections 4.3–4.5).
6. **Checked** `http://192.168.1.109:5281/api/v1/mobile/config`: it showed the new name and the portal address for phones (Figure 5.1).
7. **Moved the phone**: on the Android 14 emulator, which was signed in to the main institute, Settings → Change server. Typing `192.168.1.109:5163` (the portal) was refused; `192.168.1.109:5281` connected, showed Northfield's name, and the SuperAdmin signed in (chapter 5).

The commands are in `tools/user-manual/connection-guide/start-northfield.ps1`, which can serve as a template for another institute.

6.3 The result



Institute A: 893 students **Institute B: new and empty**

Figure 6.2 *The same app on the same phone, first on institute A, then on institute B*

Both institutes ran at the same time on the same PC and SQL Server, each with its own accounts and data. Neither installation could see the other's database.

6.4 Keeping installations apart

- **One folder per installation**: `App_Data`, `uploads` and settings files belong to one institute.
- **One database per installation**, named after the institute.

© 2026 Tabsan Nexora. All rights reserved.

- **Two ports per installation**, one for the API and one for the portal, and a firewall rule for each.
- **Environment variables per program**, set in its start script or IIS site, never machine-wide.
- **Check the first lines** of each program after every start (Figures 3.1 and 4.1).
- On a live server, give each institute its **own host name** (for example `api.northfield.edu` and `portal.northfield.edu`) behind IIS or nginx with HTTPS, instead of different ports.

© 2026 Tabsan Nexora. All rights reserved.

7 Live Installations (Production)

The example ran in the Development environment, which relaxes several checks so that one PC is enough for a trial. A live installation must run in **Production**. This is the default when `ASPNETCORE_ENVIRONMENT` is not set. In Production both programs check their settings at start-up and stop with a message that names the missing setting.

7.1 Settings the API requires in Production

Setting	Why	Example value
`JwtSettings:SecretKey`	Signs everyone's sign-in. At least 32 random characters, different for every institute.	a generated random string
`ScaleOut:RedisConnectionString`	The shared cache. Required outside Development, even on one server.	`localhost:6379`
`MediaStorage:SignedUrlSecret`	Signs download links for uploaded files.	a generated random string
`Email:Username`, `Email:Password`	Required while `NotificationEmail:Enabled` is true.	the SMTP account
`QueuePlatform:RabbitMq:ConnectionString`	Only when RabbitMQ is switched on.	the RabbitMQ address
`ReverseProxy:KnownProxies`	Required while `ReverseProxy:Enabled` is true (it is in `appsettings.Production.json`).	`127.0.0.1` (IIS or nginx on the same server)
`Deployment:TenantIsolation:…`	`appsettings.Production.json` switches tenant isolation on, which needs a `TenantCode` and a `ConfigPath` to an existing file. For one institute per installation, set its `Enabled` to `false`.	`false`
the database connection	Must be a real connection string.	chapter 2

> **IMPORTANT** Values that look like placeholders are refused: anything containing `REPLACE_WITH`, `OR_SET_VIA_ENV_VAR`, `CHANGE_ME`, `yourdomain.com`, `example.com`, `<` or `>`. Several values in the shipped `appsettings.Production.json` are such placeholders on purpose; set real values for them.

7.2 Settings the portal requires in Production

- `EduApi:BaseUrl` (or `EDUSPHERE_APP_BASE_URL`): the API's address, not a placeholder.
- `ScaleOut:SharedDataProtectionKeyRingPath`: a folder where the portal keeps the keys that protect its sign-in cookies. Several portal servers of one institute share it.

© 2026 Tabsan Nexora. All rights reserved.

- A database setting (`EDUSPHERE_DB_CONNECTION`), only to start (section 4.2).

7.3 The listening address in Production

The shipped `appsettings.Production.json` files set a fixed listening address (`Kestrel:Endpoints:Http:Url`: `http://localhost:5181` for the API, `http://localhost:5063` for the portal). An address set there **wins over** `--urls`. Change it per installation with an environment variable, or in `appsettings.Local.json`:

```
$env:Kestrel__Endpoints__Http__Url = "http://localhost:5281"
```

Behind IIS this address is not used: IIS gives the program its address.

7.4 HTTPS and a public address

- Put IIS (with the ASP.NET Core Module) or nginx in front of the API and the portal, with a certificate from a trusted authority (for example Let's Encrypt).
- Give each its own name, for example `api.northfield.edu` for the API, which people type in the app, and `portal.northfield.edu` for the portal.
- Set `Mobile:PortalBaseUrl` to the portal's https address and `EDUSPHERE_APP_BASE_URL` to the API's.
- Single sign-on and push notifications need these public https addresses as well (SSO Setup Guide; Mobile App Connection Guide).

7.5 Protect the Database Setup page

The API's `/setup` page needs no sign-in. Its **Save & Continue** and **Import & Apply Profile** buttons change the database the API uses, and **Test Connection** makes the API connect to any SQL Server someone types. Once the installation works, block every address that **starts with** `/setup` for everyone but the server itself. Do not block `setup` elsewhere in addresses: two-factor sign-in uses addresses that end in `/setup`.

IIS (URL Rewrite module), in the API site's web.config

```
<rewrite><rules>
  <rule name="Block database setup page" stopProcessing="true">
    <match url="^setup(/|$)" />
    <action type="CustomResponse" statusCode="404" />
  </rule>
</rules></rewrite>
```

nginx

```
location ^~ /setup { deny all; }
```

© 2026 Tabsan Nexora. All rights reserved.

> **WARNING** When phones reach the API directly on the institute's network (no IIS or nginx in front), every device on that Wi-Fi can open `/setup`. Put the API behind IIS or nginx with the block above, or keep that network to staff devices.

7.6 Keep your settings safe across updates

- Never edit `appsettings.json` or `appsettings.Production.json`: an update replaces them.
- Keep the institute's values in environment variables (start script or IIS) or in `appsettings.Local.json`, which updates never touch.
- Keep a copy of `web.config` (IIS) before an update.
- Back up the database, `App_Data` and `uploads` before an update.

© 2026 Tabsan Nexora. All rights reserved.

8 Troubleshooting

8.1 Database and API

What you see	Why	What to do
A new installation shows another institute's data	It found another setting first, for example a profile in `environments.json` or a machine-wide variable	Set `EDUSPHERE_DB_CONNECTION` for this program; check the line **Database connection source**
The API stops at once: "TABSAN_SUPER_PASSWORD environment variable is required for initial seeding."	First start of a new database without the variable	Set `TABSAN_SUPER_PASSWORD` (section 2.5)
Cannot sign in as superadmin after the very first start	A build from before 8 November 2026	Restart the API once
Pages redirect to `/setup`, or the API answers 503 "database_setup_required"	The API cannot reach its database	Check that SQL Server runs, the server and database names, and the account's rights; use Test Connection
"Cannot open database … requested by the login"	The database exists but the API's account has no rights on it	Give that account db_owner (section 2.2)
Every request gives error 500 after an update; `appsettings.json` is about 1 KB	An update by a build from before 8 November 2026 copied the background-jobs settings over the API's	Delete `appsettings*.json` in the folder and publish again (section 3.2)
In Production the API stops with "… must be provided by environment variables …"	A required setting is missing or a placeholder	Set it (section 7.1)
`--urls` has no effect in Production	`Kestrel:Endpoints` in `appsettings.Production.json` wins	Set `Kestrel__Endpoints__Http__Url` (section 7.3)

8.2 Portal

What you see	Why	What to do
The portal shows another institute's data	It calls another installation's API	Check the line **API address (EduApi:BaseUrl)**; set `EDUSPHERE_APP_BASE_URL`; with a build from before 8 November 2026 also `--EduApi:BaseUrl` (section 4.2)
Changes are refused: "The portal is in read-only mode …"	No valid licence	Upload the licence (section 4.5)
The sign-in page says Tabsan EduSphere, not the institute	The sign-in page always does	The name shows after sign-in and in the app

© 2026 Tabsan Nexora. All rights reserved.

What you see	Why	What to do
Signing in to one portal signs you out of another on the same computer	Cookies are kept per host name, not per port	Use different host names or browsers (section 4.3)

8.3 Mobile app

The app says	Why	What to do
"That address answers, but it is not an EduSphere server."	The portal's address or port was typed	Type the API's address (section 5.3)
"Could not reach …"	The API is not running, does not listen on the network, the firewall is closed, the server's network is Public, or the phone is on another Wi-Fi	Section 5.2; test `/api/v1/mobile/config` in the phone's browser
The right institute's name, but quizzes or sign-on open the wrong page	`Mobile:PortalBaseUrl` is wrong or says `localhost`	Set it to the portal address phones can open (section 5.1)
It refuses `http://` for the server	An internet address without https	Use https, or connect over the institute's network or VPN

All other app messages are explained in the Mobile App Connection Guide, chapter 11.

© 2026 Tabsan Nexora. All rights reserved.

Appendix A — Settings Reference

API (Tabsan.EduSphere.API)

Setting / variable	Purpose	Example
`EDUSPHERE_DB_CONNECTION`	The database (wins over every file)	chapter 2
`ConnectionStrings:DefaultConnection`	The database (settings file)	chapter 2
`App_Data\database-setup.json`	Saved by the Database Setup page; wins over everything	—
`TABSAN_SUPER_USERNAME` / `_PASSWORD` / `_EMAIL`	The first SuperAdmin of a new database	`superadmin`
`--urls` (or `ASPNETCORE_URLS`)	Listening address (Production: section 7.3)	`http://0.0.0.0:5281`
`ASPNETCORE_ENVIRONMENT`	`Development` (trial) or `Production` (live; the default)	`Production`
`Mobile:PortalBaseUrl`	Portal address for phones	`http://192.168.1.109:5163`
`Licensing:EnforceReadOnlyWhenUnlicensed`	Read-only without a licence (default `true`)	trials only: `false`
`JwtSettings:SecretKey`, `ScaleOut:RedisConnectionString`, `MediaStorage:SignedUrlSecret`	Required in Production	chapter 7

Portal (Tabsan.EduSphere.Web)

Setting / variable	Purpose	Example
`EDUSPHERE_APP_BASE_URL`	The API the portal calls (wins over every file)	`http://localhost:5281`
`EduApi:BaseUrl`	The same, in a settings file	`http://localhost:5181`
`EDUSPHERE_DB_CONNECTION`	Needed only to start	the API's string
`--urls`	Listening address in Development	`http://0.0.0.0:5163`
`ScaleOut:SharedDataProtectionKeyRingPath`	Key folder; required in Production	`D:\EduSphere\keys`

© 2026 Tabsan Nexora. All rights reserved.

Appendix B — Addresses, Ports and the App

Mobile app settings

Setting	Purpose	Example
Server address (typed once)	The institute's API	`192.168.1.109:5281`, `api.northfield.edu`
Branded build: `ApiBaseUrl`, `LockServer`	Server built into the app	Mobile App Connection Guide, chapter 3

Addresses to check

Address	What it shows
`http://<server>:5281/health`	**Healthy** when the API and its database work
`http://<server>:5281/api/v1/mobile/config`	The institute's name and portal address: what the app reads first
`http://<server>:5281/setup`	The Database Setup page (block it on live servers, section 7.5)
`http://<server>:5163/`	The portal's sign-in page

Ports used in the example

Installation	API	Portal	Database
Institute A (main)	5181	5063	`Tabsan-EduSphere`
Institute B (Northfield)	5281	5163	`EduSphere-Northfield`

© 2026 Tabsan Nexora. All rights reserved.

Appendix C — Document History

Version	Date	Changes
1.0	8 November 2026	First edition: connecting database, API, portal and app; worked example of a second institute on one server; Production settings; troubleshooting.

© 2026 Tabsan Nexora. All rights reserved.